



ITI · INTRODUCTION TO PYTHON TRACK

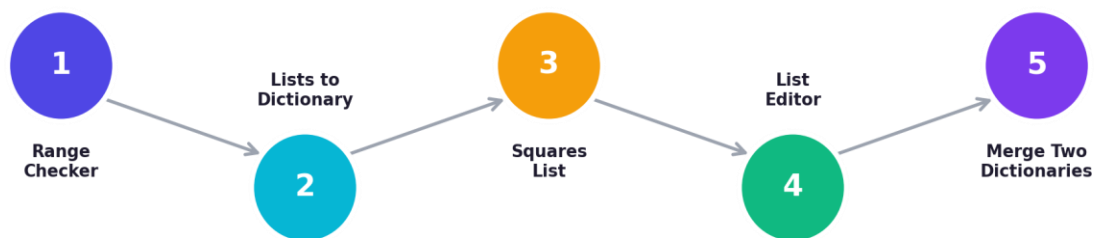
Python Lab 2

Lists, Dictionaries & Scope in Practice

Hands-on exercises applying today's list/dictionary methods to small, realistic problems.

Field	Detail
Instructor	Fatma Elraey
Track	Python for AI Applications
Session	Lab 2 · Day 2 of 6
Focus	Ranges · list-to-dict conversion · comprehensions · list editing · dictionary merging

Roadmap





Exercises

01

Range Checker

Write a Python function to check whether a number falls in a given range. For example, checking the range (-5, 5) with a number inside it should return True.

- Define a function like `in_range(num, start, end)`.
- Use a comparison chain: `start <= num <= end`.
- Test it with a number inside the range and one outside.

AI connection: This is the same logic used to validate that a model's confidence score or a sensor reading falls within an expected, trustworthy range.

02

Two Lists Into a Dictionary

Write a Python program to convert two lists into a dictionary, where each item from list1 becomes a key and the item at the same position in list2 becomes its value.

- Example: `list1 = ['a','b','c'], list2 = [1,2,3] -> {'a':1, 'b':2, 'c':3}`
- Look up Python's built-in `zip()` function — it pairs up items from two lists by position.
- `dict()` can build a dictionary directly from pairs.

AI connection: This mirrors turning a list of feature names and a list of feature values into a single labeled record — exactly the shape most ML models expect as input.

03

List of Squares

Write a Python function that creates and prints a list containing the square of every number from 1 to 30 (both included).

- A basic loop with `.append()` works, or try a list comprehension: `[x**2 for x in range(1, 31)]`
- Remember `range(1, 31)` is needed to include 30 — `range`'s end is exclusive.

AI connection: Generating derived numeric sequences like this is the everyday building block behind feature engineering — squaring, scaling, or transforming raw values before training.

04

Editing a List Step by Step

Write a program that takes the list `[3, 6, 4, 0, 8]` and then: (a) removes the last element, (b) adds 'R' in the second place, (c) asks the user to input a specific number in the list, then deletes it by its VALUE (not its index).



- (a) Think about which method removes the last item without needing to know its index.
- (b) `.insert(index, value)` lets you place an item at a specific position.
- (c) Use `input()` to read the number, convert it with `int()`, then remove it by value — not by position.

AI connection: This kind of step-by-step list surgery is exactly what you do when cleaning a raw dataset: dropping a bad row, inserting a placeholder, or removing an outlier by its value.

05

Merge Two Dictionaries

Create two dictionaries and merge them into one. Take into consideration that keys should be unique in the final result.

- Decide what should happen if both dictionaries share a key — which value should win?
- Yesterday's dictionary method `.update()` is built exactly for this.
- The unpacking syntax `{**dict1, **dict2}` is another common way to merge two dictionaries.

AI connection: Merging dictionaries is exactly how you'd combine a default model configuration with a user's custom settings before starting a training run.

✓ QUICK CHECK Before You Start

Q1. In Exercise 2, what does `zip(list1, list2)` actually produce?

Answer: An iterator of paired tuples, e.g. `zip(['a','b'],[1,2])` yields `('a',1)` then `('b',2)` — perfect for feeding straight into `dict()`.

Q2. In Exercise 4c, why remove by value instead of by index?

Answer: The user types a number they want gone, not a position — `.remove(value)` searches for and deletes the first matching item, which is what removing 'by value' means.

Q3. In Exercise 5, if both dictionaries have the key 'name', which value wins after merging with `.update()`?

Answer: The value from the dictionary passed INTO `update()` — it overwrites the value already in the dictionary being updated.



Stretch Goal

If you finish early, extend Exercise 2 so it also works in reverse — write a function that takes a dictionary and returns two separate lists: one of its keys and one of its values, in matching order.

```
def dict_to_lists(d):  
    keys = list(d.keys())  
    values = list(d.values())  
    return keys, values  
  
k, v = dict_to_lists({'a': 1, 'b': 2, 'c': 3})  
print(k)    # ['a', 'b', 'c']  
print(v)    # [1, 2, 3]
```

AI CONNECTION *Splitting a dictionary back into aligned key/value lists is a common step when preparing labeled data for a plotting library or a training loop that expects separate arrays.*

Nice work! Lists and dictionaries are the two data structures you'll reach for most often — next we'll build on them with Python's Object-Oriented tools.